

Organización de Datos

Resumen Integral de la Unidad 1

Ingeniería Informática · Cátedra GDI

Este documento constituye un resumen académico de estudio autónomo basado en la presentación introductoria de la materia. Desarrolla todos los conceptos fundamentales de forma explicativa y articulada, con ejemplos prácticos y notas de advertencia.

Tabla de Contenidos

1. Introducción a la Organización de Datos
2. Conceptos Fundamentales: Dato, Registro y Campo
3. Archivo de Datos
4. Clasificación de Archivos por Función
5. Clasificación por Tipo de Datos y de Registros
6. Clasificación por Tipo de Acceso
7. El Sistema Operativo y el Sistema de Archivos
8. Visión Lógica de Archivos: Atributos y Operaciones
9. Organización de Archivos: Concepto y Modos
10. Diseño de Datos: Conceptual y Lógico
11. Tipos de Atributos e Identificadores
12. Definición Lógica de Datos y Tipos de Valores
13. Organización de Registros: Longitud Fija vs Variable
14. Problemática de los Registros de Longitud Variable
15. Independencia Lógica de Datos
16. Primitivas de Organización de Archivos

1. Introducción a la Organización de Datos

La **Organización de Datos** es la rama de la informática que estudia las técnicas de representación y almacenamiento de datos en archivos, con el objetivo central de permitir su **recuperación, resguardo y transmisión de forma eficiente y segura**. En un mundo donde los sistemas de información manejan volúmenes enormes de datos, elegir correctamente cómo organizar esos datos es una decisión de diseño crítica: impacta directamente el rendimiento de las operaciones, el uso del espacio en disco y la robustez ante fallos.

La materia aborda estas técnicas comenzando por las bases conceptuales —qué es un dato, cómo se estructura lógicamente, qué es un archivo— y luego profundiza en los algoritmos y estructuras que permiten localizar, insertar, modificar o eliminar registros de manera óptima. Todas las definiciones y diseños son **independientes del lenguaje de programación** utilizado, lo que le confiere a este conocimiento un carácter universal y transferible.

2. Conceptos Fundamentales: Dato, Registro y Campo

Para comprender la organización de archivos es imprescindible precisar tres conceptos que forman la columna vertebral de toda la materia: el **dato**, el **registro** y el **campo**.

El Dato

Un dato es la representación de una *cosa real o ideal*, o de un *evento ocurrido o programado para ocurrir*. Esta definición es deliberadamente amplia: puede tratarse de un producto comercial, una persona, una transacción bancaria o una reserva futura. Lo fundamental es que el dato existe en el sistema de información como una **unidad lógica de manipulación**, denominada **registro**.

Cada registro describe al objeto o evento a través de sus **características descriptivas**, también llamadas **atributos**. Por ejemplo, una persona se describe por su nombre, apellido, fecha de nacimiento, domicilio, etc. Cada uno de esos atributos se almacena en una unidad más pequeña llamada **campo**. La jerarquía es entonces: el archivo contiene registros, y cada registro contiene campos.

■ **Nota:** La distinción entre *atributo* (característica abstracta) y *campo* (representación concreta en el archivo) es análoga a la distinción entre clase y atributo de instancia en programación orientada a objetos.

3. Archivo de Datos

Un **archivo de datos** es la **unidad lógica de almacenamiento permanente de registros**, administrada por el Sistema Operativo. La palabra «permanente» es clave: a diferencia de la memoria RAM, el archivo persiste aun cuando el sistema se apaga. El Sistema Operativo es el intermediario entre los programas —que ven los datos en términos lógicos— y los dispositivos

físicos de almacenamiento.

Dentro de un archivo, los registros pueden agruparse en unidades lógicas de mayor tamaño llamadas **bloques** o **páginas**. Esta agrupación responde a una razón de eficiencia: los dispositivos físicos transfieren datos por bloques (típicamente múltiplos de 512 bytes), por lo que leer varios registros en una sola operación física —en lugar de uno por vez— reduce drásticamente el tiempo de acceso a disco.

4. Clasificación de Archivos por Función

Los archivos pueden clasificarse según el rol que cumplen dentro de un sistema de información. Esta taxonomía funcional ayuda al diseñador a elegir la estructura y el modo de acceso más adecuados para cada caso de uso.

Maestros	Contienen datos que representan entidades de existencia real o ideal del sistema: productos, clientes, empleados. También incluyen tablas de referencia (dominios de atributos definidos por extensión), como la lista de provincias o de monedas válidas. Son los datos «estables» del sistema.
Transaccionales	Registran hechos o eventos relacionados con los datos maestros: ventas de productos, prestaciones de servicios, movimientos de cuenta. Crecen continuamente con el tiempo y suelen requerir estrategias de depuración y respaldo periódicos.
Reporte	Contienen información ya procesada y editada para presentarla al usuario. Generalmente no se modifican una vez generados y se distribuyen en formatos PDF, HTML o texto plano.
Trabajo	Almacenan resultados parciales o intermedios de un proceso, o datos que se intercambian entre programas en una pipeline de procesamiento. Su vida útil es corta: existen mientras dura el proceso que los usa.
Control de Datos	Guardan metadatos (definiciones de datos), administran espacios libres, registran identificadores de registros vacantes o sirven como índices y tablas de acceso a otro archivo. Son la «infraestructura» del sistema de archivos.
Intercambio de Datos	Representan datos en formatos estándar y abiertos (como XML) de modo que cualquier sistema que conozca el estándar pueda procesarlos, independientemente de la plataforma o lenguaje. Suelen ser archivos de texto con convenciones de rotulado o delimitación.
Recursos de Programa	También llamados «unidades grandes de información»: imágenes, audio, video. Su particularidad es el enorme tamaño individual de cada registro y la naturaleza binaria y no estructurada de su contenido.
Productos de Programas	Archivos con un tipo asociado a una aplicación específica: .docx, .xlsx, .psd. Su estructura interna es conocida y explotada únicamente por el programa propietario —salvo que existan especificaciones públicas del formato.

Empaquetado de Archivos	Agrupar y comprimen archivos y directorios para transmisión o respaldo (.zip, .rar, .tar.gz). No representan datos del dominio de la aplicación, sino que son contenedores de otros archivos.
--------------------------------	---

5. Clasificación por Tipo de Datos y de Registros

Por tipo de datos almacenados

Los archivos también se clasifican por la naturaleza de los datos que contienen. Los archivos de **texto** (ASCII o Unicode) almacenan caracteres legibles por humanos; cualquier editor de texto puede abrirlos. Los archivos **binarios** almacenan secuencias de bytes cuya interpretación depende del contexto: pueden representar enteros en complemento a dos, números de punto flotante, imágenes comprimidas, etc. Dentro de los binarios, los **tipificados** tienen una estructura conocida por el SO o algún estándar, mientras que los archivos **sin tipo** son secuencias de bytes sin interpretación predefinida.

Por tipo de registros

Longitud Fija	Todos los registros ocupan exactamente la misma cantidad de bytes. Esto simplifica enormemente el cálculo de posiciones: dado el número de registro n y el tamaño fijo L , la posición en bytes es $n \times L$. La contrapartida es el posible desperdicio de espacio cuando los campos no se utilizan completamente.
Longitud Variable	Los registros pueden tener distintos tamaños, lo que aprovecha mejor el espacio pero complica la localización y extracción de registros individuales. Se emplean mecanismos como prefijos de longitud, delimitadores especiales o tablas de índice para poder navegar el archivo.

6. Clasificación por Tipo de Acceso

El tipo de acceso define *cómo* el programa llega a un registro particular dentro del archivo. Esta clasificación es especialmente relevante al diseñar algoritmos de búsqueda y al elegir la estructura de almacenamiento.

Secuencial	Los registros se leen o escriben en orden, uno a continuación de otro, desde el inicio hasta el final del archivo. Es el único modo posible en ciertos medios (cintas magnéticas) y el más eficiente para procesar la totalidad de los registros.
Directo	Permite acceder a cualquier registro por su posición física (número de bloque o dirección de byte) sin recorrer los anteriores. Se implementa mediante la operación seek del SO, que desplaza el puntero de lectura/escritura a una posición arbitraria.
Indexado	Los registros se almacenan secuencialmente, pero existe un archivo auxiliar de índice que mapea valores de clave a posiciones físicas, permitiendo localizar un registro por su contenido sin recorrer todo el archivo.

Relativo	Similar al directo, pero la dirección se calcula a partir de un número relativo de registro (RRN, Relative Record Number), típicamente usando la clave del registro y una función de transformación (hashing).
Stream	El archivo se trata como un flujo continuo de bytes, sin estructura de registros. El programa lee o escribe la cantidad de bytes que necesite. Es el modelo más bajo nivel y el más flexible.

7. El Sistema Operativo y el Sistema de Archivos

El **Sistema Operativo (SO)** es el responsable del manejo de todos los recursos de la computadora, tanto **físicos** (hardware: disco, CPU, memoria) como **lógicos** (carpetas y archivos). Su rol clave es proveer una **interfaz controlada** entre los usuarios/aplicaciones —que trabajan con una visión lógica de la información— y los dispositivos de almacenamiento secundario —que operan a nivel físico con bloques de bytes.

El conjunto de programas del SO encargados específicamente de esta mediación se denomina **Sistema de Archivos**. Es el Sistema de Archivos quien traduce una operación de alto nivel como «leer el registro con DNI 12345678 del archivo personas.dat» en una serie de operaciones físicas sobre el disco: calcular el sector, transferir el bloque al buffer en RAM, entregar los bytes al programa.

Funciones principales del SO respecto a archivos

Identificación y Ubicación	Mantiene el directorio (estructura jerárquica o catálogo) que mapea nombres lógicos de archivos a su ubicación física en el dispositivo.
Seguridad	Gestiona usuarios y grupos, y controla los permisos de acceso a los datos (lectura, escritura, ejecución).
Asignación de espacio	Administra qué sectores del disco ocupa cada archivo y lleva un registro de las unidades de asignación libres.
Coordinación de transferencia	Atiende y prioriza las solicitudes simultáneas de lectura/escritura de múltiples procesos, evitando conflictos.
Concurrencia	Administra el acceso concurrente de varios procesos a un mismo registro físico, implementando mecanismos de bloqueo para preservar la integridad de los datos.

8. Visión Lógica de Archivos: Atributos y Operaciones

Desde el punto de vista lógico —el que perciben los usuarios y las aplicaciones— un archivo es una entidad con **atributos** propios y sobre la que se pueden ejecutar **operaciones**.

Atributos de un archivo

Los metadatos que el SO mantiene por cada archivo incluyen: **nombre** (identificador legible), **tamaño** (en bytes u otras unidades), **fecha y hora** de creación/modificación/acceso, **dueño**

(usuario propietario), **grupo** al que pertenece, e **indicadores** (flags) que señalan propiedades como «sólo lectura», «oculto», «de sistema», etc.

Operaciones sobre archivos

El SO expone una interfaz de operaciones estándar que las aplicaciones pueden usar sin necesidad de conocer el hardware subyacente:

Crear / Eliminar	Crean un nuevo archivo (reservando espacio) o lo eliminan liberando el espacio asignado.
Abrir / Cerrar	Abrir prepara el archivo para uso (carga metadatos, reserva buffers). Cerrar libera esos recursos.
Leer / Escribir	Transfieren datos entre el archivo y la memoria del proceso. La lectura avanza el puntero de posición.
Escribir al final	Operación especial de append que siempre agrega datos al final, sin necesidad de conocer el tamaño actual.
Posicionarse (seek)	Mueve el puntero de lectura/escritura a una posición arbitraria, habilitando el acceso directo.
Obtener / Definir atributos	Consultan o modifican los metadatos del archivo (permisos, timestamps, etc.).
Renombrar	Cambia el nombre lógico del archivo sin mover sus datos físicos.

9. Organización de Archivos: Concepto y Modos

La **organización de un archivo** define dos cosas fundamentales: *dónde almacenar los registros nuevos* y *cómo encontrar registros existentes* para eliminarlos, modificarlos o recuperarlos. En palabras más formales, es «el modo de disponer los registros del fichero en el soporte».

El objetivo de la materia es precisamente optimizar estas decisiones: elegir la organización más adecuada puede marcar la diferencia entre un sistema que responde en milisegundos y uno que tarda minutos en encontrar un registro.

Los tres modos principales de organización

Secuencial	Los registros se disponen uno a continuación del otro en el orden en que fueron insertados (o en un orden predefinido, como orden alfabético o numérico por clave). Es la forma más simple. La búsqueda de un registro particular requiere en promedio recorrer la mitad del archivo, lo que puede ser costoso para archivos grandes.
Directo	Los registros se ubican en posiciones calculadas a partir de su clave, mediante un algoritmo de transformación (función de hashing). Esto permite localizar cualquier registro en tiempo constante —idealmente con una sola operación de seek— sin necesidad de recorrer el archivo.

Indexado	Los registros se almacenan típicamente en forma secuencial, pero se mantiene un archivo auxiliar de índice que relaciona cada valor de clave con la posición física del registro correspondiente. La búsqueda primero consulta el índice (pequeño y fácil de recorrer) y luego va directamente a la posición indicada.
-----------------	--

10. Diseño de Datos: Conceptual y Lógico

Antes de implementar cualquier archivo, es necesario diseñar los datos que contendrá. Este diseño se realiza en dos etapas sucesivas y complementarias, ambas **independientes del lenguaje de programación** que se utilizará para la implementación.

Diseño Conceptual (de alto nivel)

En esta etapa se definen los **atributos** del conjunto de datos —es decir, qué características hay que representar— sin especificar aún tipos de datos ni dominios concretos. El resultado es una definición abstracta que describe la realidad del dominio del problema. Por ejemplo, se decide que una *Persona* tiene DNI, apellido(s), nombre(s), fecha de nacimiento, domicilio(s), correos electrónicos y número de celular; pero todavía no se especifica si el DNI será un entero de 4 bytes o una cadena de 8 caracteres.

La definición conceptual también establece qué atributos son **identificadores** (claves) y cuáles son opcionales, compuestos o polivalentes. Esta información se expresa mediante una notación formal con **calificadores**.

Diseño Lógico (de programación o de bajo nivel)

En esta segunda etapa se toman las decisiones concretas de implementación: se asigna un **tipo de dato** y un **dominio** a cada atributo, y se define la **convención de almacenamiento y recuperación** dentro del archivo. El resultado es la especificación completa de la estructura del registro, lista para ser codificada en cualquier lenguaje.

11. Tipos de Atributos e Identificadores

Tipos de Atributos

Simple	Representa una única propiedad indivisible. Ejemplos: Nombre, Apellido, Peso, Altura.
Compuesto	Está formado por sub-atributos. Ejemplo: Fecha de nacimiento, compuesta por día, mes y año. En el archivo puede almacenarse como un bloque o como campos separados.
Opcional	Puede o no estar presente en un registro concreto. Ejemplo: número de celular (no toda persona lo tiene). Requiere un mecanismo para indicar su ausencia (valor nulo).

Polivalente	Puede tomar múltiples valores del mismo tipo para una misma entidad. Ejemplo: una persona puede tener varios correos electrónicos. Requiere indicar cuántos valores hay.
Externo	Relaciona un dato con otro u otros del mismo conjunto de datos o de otro conjunto. Es el equivalente a una clave foránea en bases de datos relacionales.

Identificadores y Calificadores

Cada entidad debe poder distinguirse unívocamente de las demás. Para ello, uno o más atributos actúan como **identificador** o clave. Un mismo conjunto de datos puede tener más de un identificador posible (claves candidatas). Por ejemplo, para *Persona* existen al menos tres: el DNI por sí solo, la combinación de apellidos+nombres, y el correo electrónico.

La notación de la cátedra emplea **calificadores** para documentar estas propiedades de forma compacta:

Calificador	Símbolo	Significado
Identificador	<u>i</u> (subrayado)	Atributo que identifica unívocamente a la entidad
Identificador externo	ie	Clave que referencia a otra entidad
Definido por extensión en otro archivo	d	El dominio está en un archivo maestro auxiliar
Opcional	?	Cardinalidad 0 o 1 (puede estar ausente)
Ninguno o varios	*	Cardinalidad 0 a N
Uno o varios	+	Cardinalidad 1 a N (obligatorio, puede repetirse)

La definición conceptual completa de *Persona* según la notación de la cátedra quedaría:

```
Persona( (dni)i, ((apellido)+, (nombres)+)i, (fecha de nacimiento(mes, dia, año)), (domicilio(tipo('Real'|'Legal'), calle, numero, piso, localidad, (provincia)d, (c.postal)?))* , (mail)*, (num celular)? )
```

■ **Nota:** En esta notación, el subrayado en **i** indica identificador, el **+** significa «uno o varios», el ***** indica «cero o varios», el **?** indica «opcional», y el **d** indica dominio definido en otro archivo.

12. Definición Lógica de Datos y Tipos de Valores

Una vez completado el diseño conceptual, el diseño lógico agrega la información necesaria para la implementación concreta: **tipo de dato** y **tamaño** para cada atributo, y la determinación de si el registro será de longitud fija o variable. Esta etapa es igualmente independiente del lenguaje de programación.

Tipos de valores convencionales

Tipo	Notación	Descripción
Entero	En	Entero en complemento a dos, almacenado en n bytes (E1=1 byte, E2=2 bytes, E4=4 bytes)
Fraccionario	Fn	Número de punto flotante IEEE 754 en n bytes (F4=simple precisión, F8=doble precisión)
Caracteres	Cn	Cadena de exactamente n caracteres (longitud fija)
Caracteres Variables	CV	Cadena de hasta 255 caracteres; se almacena con un prefijo de longitud de 1 byte
Texto	T	Cantidad ilimitada de caracteres, incluyendo caracteres de control (salto de línea, retorno de carro)
Lógico	L	Valor booleano: 0=Falso/No, 1=Verdadero/Sí. Ocupa 1 byte
Binario	B	Dato binario arbitrario: imagen, audio, video, etc.

Ejemplo comparativo: longitud fija vs. variable

Para ilustrar la diferencia, la siguiente tabla muestra cómo se especificaría el registro de *Persona* en ambas variantes:

Atributo	Longitud Fija	Longitud Variable
dni	E4 (4 bytes)	E4 (4 bytes)
apellido	C64 (64 bytes fijos)	CV (hasta 255, con prefijo)
nombres	C64 (64 bytes fijos)	CV (hasta 255, con prefijo)
mes/dia/año	E1/E1/E2	E1/E1/E2
domicilio (hasta 3)	C64×3 campos × 3 ocurrencias fijas	cantDoms:E1 + CV por campo
mail (hasta 2)	C32 × 2 fijos	cantMail:E1 + CV por ocurrencia
num celular	C16 (o vacío con padding)	cantCel:E1 + CV opcional

13. Organización de Registros: Longitud Fija vs. Variable

Registros de Longitud Fija

En un archivo de registros de longitud fija todos los registros ocupan exactamente la misma cantidad de bytes. La información se almacena con un **encabezado (header)** al inicio del archivo que describe la estructura, seguido de los registros ubicados en posiciones consecutivas.

La principal ventaja es la **simplicidad del acceso directo**: dado el número de registro *n* (comenzando desde 0) y el tamaño *L* del registro en bytes, la posición de inicio del registro dentro del archivo es exactamente:

$$\text{posición} = \text{tamaño_encabezado} + n \times L$$

Esta fórmula permite implementar el acceso directo con una única operación de seek, sin necesidad de recorrer los registros anteriores. La contrapartida es que cuando los campos no

se utilizan en su totalidad (por ejemplo, un apellido de 5 letras en un campo de 64 caracteres), los bytes sobrantes se desperdician como «relleno» o padding.

Registros de Longitud Variable

Un archivo de registros de longitud variable puede almacenar elementos de tipos distintos y permite que cada registro ocupe el espacio que realmente necesita. Esto es especialmente valioso cuando hay atributos opcionales o polivalentes, ya que los registros con pocos valores de esos atributos serán notablemente más cortos.

La **utilización del espacio es más eficiente**, pero a cambio la localización y extracción de registros individuales se complica: para llegar al registro número n es necesario haber «leído» (al menos en cuanto a longitud) todos los registros anteriores, o disponer de un índice auxiliar que registre la posición de inicio de cada uno.

■ **Nota:** La longitud de cada registro debe estar definida de forma inequívoca para que el lector pueda determinar dónde termina un registro y comienza el siguiente. Las dos estrategias más habituales son: (a) incluir un campo de longitud al inicio de cada registro, y (b) usar un delimitador especial (carácter centinela) al final de cada registro.

14. Problemática de los Registros de Longitud Variable

Los registros de longitud variable introducen varios desafíos de implementación que deben abordarse explícitamente en el diseño del archivo:

Representación de valores nulos

Los atributos opcionales (calificador ?) pueden estar ausentes en ciertos registros. La solución habitual son los **mapas de nulidad (null bitmaps)**: un campo al inicio del registro con un bit por cada atributo opcional. Si el bit está en 0, el atributo está ausente y su campo no ocupa espacio en ese registro.

Factor de Bloqueo

El **factor de bloqueo** (FB) indica cuántos registros caben en un bloque del disco. Los bloques tienen tamaños que son potencias de dos multiplicadas por 512 bytes (512 B, 1 KB, 2 KB, 4 KB, 8 KB...), coordinando con el sistema de buffering del disco.

Ejemplo: si el tamaño de bloque es 4 KB (4096 bytes) y cada registro ocupa 1,5 KB (1536 bytes), el factor de bloqueo es:

$$FB = \text{floor}(4096 / 1536) = \text{floor}(2,67) = 2 \rightarrow \text{caben 2 registros por bloque, con 1024 bytes de desperdicio interno por bloque (fragmentación interna).}$$

Administración del espacio libre

Cuando se elimina un registro, el espacio que ocupaba queda disponible para reutilización. Dos estrategias principales:

Encadenamiento de registros libres (longitud fija)	Los registros eliminados se enlazan en una lista (free list). El encabezado del archivo apunta al primero de los registros libres, y cada uno apunta al siguiente. La inserción toma el primer registro libre de la lista. Simple y eficiente.
Mapas de bits de espacio libre (longitud fija)	Un mapa de bits en el encabezado tiene un bit por cada posible registro: 1 si está ocupado, 0 si está libre. La búsqueda de espacio libre es una operación de bit a bit.
Desperdicio controlado (longitud variable)	El espacio liberado por un registro eliminado puede no reutilizarse de inmediato —porque un nuevo registro de diferente tamaño podría no caber exactamente— generando fragmentación. La solución alternativa es un archivo auxiliar que registra los «huecos» disponibles con su tamaño y posición.

■ **Advertencia:** La fragmentación interna (bytes sobrantes dentro de un bloque) es característica de la longitud fija. La fragmentación externa (huecos dispersos en el archivo) es el problema típico de la longitud variable. Ambas reducen la eficiencia de almacenamiento y, en casos extremos, requieren una operación de reestructuración del archivo.

15. Independencia Lógica de Datos

Una dimensión de diseño importante es el grado en que la **estructura interna del archivo** es conocida o accesible por agentes externos a la aplicación que lo creó.

Datos lógicamente Dependientes	La estructura y organización de los registros sólo la conoce la aplicación propietaria (está «hardcodeada» en el código). Ningún otro programa puede interpretar el archivo sin acceder al código fuente. Es el esquema más frágil: un cambio en la estructura rompe todos los programas existentes.
Datos lógicamente Independientes	Existe una definición pública de los datos (metadatos) en una convención estándar —como XML, JSON o un esquema de base de datos—. Cualquier programa que conozca el estándar puede leer e interpretar el archivo. Este diseño favorece la interoperabilidad y la mantenibilidad.

Los metadatos pueden incorporarse al archivo de dos formas: como **encabezados dentro del propio archivo** (cada archivo es autodescriptivo) o en un **diccionario de datos externo** (un repositorio centralizado de definiciones). La primera opción es más portable; la segunda facilita la gestión centralizada.

16. Primitivas de Organización de Archivos

Las **primitivas** son las operaciones de alto nivel que cualquier sistema de archivos bien diseñado debe proveer. Se organizan en cuatro grandes grupos:

De Creación	Creación y carga inicial del archivo. En esta etapa no se valida unicidad de claves ni se busca espacio libre: se asume que los datos de entrada son correctos y se insertan secuencialmente. El objetivo es la máxima velocidad de carga.
De Actualización de Registros	Comprende tres operaciones: Inserción (agrega un nuevo registro, validando que la clave no exista ya y buscando un espacio libre apropiado), Modificación (localiza el registro y actualiza uno o más campos) y Supresión (elimina el registro, liberando el espacio o marcándolo como disponible).
De Recuperación de Registros	Dos modalidades: Consulta unitaria (recupera un único registro por su clave o criterio) y Reporte o recuperación comprensiva (recupera un conjunto de registros que cumplen ciertos criterios, típicamente recorriendo todo o gran parte del archivo).
De Mantenimiento	Operaciones periódicas necesarias para mantener la salud del archivo: Reestructuración o reconstrucción (elimina la fragmentación, compacta el espacio libre, actualiza índices), Depuración (en archivos transaccionales, elimina registros obsoletos o muy antiguos) y Respaldo (copia de seguridad para recuperación ante fallos).

■ **Advertencia:** Las primitivas de creación y las de actualización tienen comportamientos muy diferentes. Durante la carga inicial se omiten las validaciones de unicidad por razones de performance; pero en las operaciones de inserción posteriores, omitir esa validación causaría inconsistencias en el archivo (claves duplicadas). Nunca confundir ambas etapas.